Assignment 1 - Test Plan - Matcha Capstone

Part 1. Description of Overall Test Plan

For this project, we were tasked with building an enterprise-grade web application for a banking reconciliation system that would handle receipts and bank statement matching. As a result, we aim to produce a piece of software that is both resilient and capable of handling large amounts of requests and processing needs. For this test plan, we devise test cases in 3 different aspects of this project: AI pipeline (AIP), Backend (BE), Frontend (FE).

In AI pipeline test cases, we ensure that our model is reliable and resilient to out of domain input. This includes making sure that server is healthy for 99% of the time (fast response time), model confidently rejecting non-receipt input and output human in loop token if unsure about a text instead of hallucinating. In addition, the model consistently producing correct structured output is crucial in the next stage where data is processed and saved to the database.

In Backend test cases, we ensure that the workflow is correct, robust, and reliable. The backend is responsible for importing bank statement records and receipts, ingesting data produced by the AI pipeline, and interacting with the databases to store and retrieve reconciliation results.

In Frontend test cases, we ensure that the app is visually clear and functional. Ideally, the page should convey all of its functionality just from a quick glance, and all the buttons and UI elements should be responsive, easily identifiable and work as intended.

Part 2. Test Case Descriptions

AI Pipeline - AIP (AI model for parsing receipts)

AIP 1.1 - Vision Language Model (VLM) Structured Output Test
AIP 1.2 - The purpose of this test is to make sure the output from VLM are structured yaml code output.
AIP 1.3 - Given a set of receipts, call the VLM and get output. Parse the output yaml code and verify with ground truth data.
AIP 1.4 - Inputs: The receipts, in form of scanned images or pdf-2-imgs
AIP 1.5 - Outputs: Structured yaml code parsed and compared against ground truth, human verified parsed data.
AIP 1.6 - Normal
AIP 1.7 - Whitebox
AIP 1.8 - Functional
AIP 1.9 - Unit

AIP 2.1 - Vision Language Model (VLM) Time To First Token Heath Check Test
AIP 2.2 - The purpose of this test is to make sure the health check API call to VLM model server will response in less than 1 seconds
AIP 2.3 - Using a single prompt with no other multi modal output, we calculate the time since we make the API call to the time the first token was received. Test pass if time < 1 seconds - healthy server.
AIP 2.4 - Inputs: The single prompt: “Quantum Mechanic in 2 sentences.” to be sent to the VLM server. No other input.
AIP 2.5 - Outputs: Result of the elapsed time since making API call and receiving the first token.
AIP 2.6 - Normal
AIP 2.7 - Whitebox
AIP 2.8 - Performance
AIP 2.9 - Unit

AIP 3.1 - Vision Language Model (VLM) Deterministic Response Test
AIP 3.2 - The purpose of this test is to make sure given a receipt, the model always output the same response every time without any deviation,
AIP 3.3 - Using a receipt image input, make 5 sequential API calls to parse the receipt. The responses should be saved, parsed and checked against each other for total identicality.
AIP 3.4 - Inputs: A single receipt image + parsing instruction prompt. Use this same input for other API calls.
AIP 3.5 - Outputs: The 5 responses from 5 identical API calls. Cross checked should show no deviation.
AIP 3.6 - Normal
AIP 3.7 - Whitebox
AIP 3.8 - Functional
AIP 3.9 - Unit

AIP 4.1 - Vision Language Model (VLM) Non-Receipt Input Test
AIP 4.2 - The purpose of this test is to make sure given an image that is not a receipt, the model does not hallucinate and force receipt content out of arbitrary information and instead output an error message.
AIP 4.3 - Using an image that is not a receipt, make the receipt parsing API call to the model. The model should output an error message
AIP 4.4 - Inputs: Images from one of the categories (certificate, ad poster, piece of literature, newspaper, …) + receipt parsing instruction as usual
AIP 4.5 - Outputs: The response from the model indicating non-receipt input error message.
AIP 4.6 - Abnormal
AIP 4.7 - Whitebox
AIP 4.8 - Functional
AIP 4.9 - Unit

AIP 5.1 - Vision Language Model (VLM) Blurry illegible Receipt Input Test

AIP 5.2 - The purpose of this test is to make sure that given a blurry, non-legible receipt (even by human), the model should confidently output “human-advise-needed” error response instead of hallucinate numbers from blurry, non-legible text.
AIP 5.3 - Using a receipt image that is blurry and non-legible, make an API call to try to parse the receipt. The response should be “human-advise-needed” error response.
AIP 5.4 - Inputs: A single receipt blurry, non-legible image + parsing instruction prompt.
AIP 5.5 - Outputs: The model response with “human-advise-needed” error.
AIP 5.6 - Abnormal
AIP 5.7 - Whitebox
AIP 5.8 - Functional
AIP 5.9 - Unit

Backend - BE (Bank Reconciliation Workflow)

BE 1.1 - Bank Statement / Receipt Import
BE 1.2 - Purpose of test: Verify that the backend can successfully import and store the bank statements and receipts, and forward them to the AI pipeline
BE 1.3 - Description of test: Upload bank statement and receipt data, and verify that it is sent to the AI pipeline for processing
BE 1.4 - Inputs used for test: Receipts and bank statements
BE 1.5 - Expected outputs/results: Receipt and bank statements objects are stored in the database, and a request is sent to the AI pipeline
BE 1.6 - Normal
BE 1.7 - Black-box
BE 1.8 - Functional
BE 1.9 - Integration

BE 2.1 - Receipt Data Ingestion from AI Pipeline
BE 2.2 - Purpose of test: Verify that the backend can successfully ingest structured receipt data returned from the AI pipeline and store it correctly
BE 2.3 - Description of test: Get parsed receipt data (vendor, date, amount, invoice number) from the AI pipeline and store it correctly in the database
BE 2.4 - Inputs used for test: JSON object containing receipt ID, vendor, date, amount, etc.
BE 2.5 - Expected outputs/results: Receipt data is stored correctly in the database with a reference field linking to the corresponding receipt object, and the request returns a success response
BE 2.6 - Normal
BE 2.7 - Black-box
BE 2.8 - Functional
BE 2.9 - Integration

BE 3.1 - Unmatched Receipt Handling
BE 3.2 - Purpose of test: Verify that receipts not matched to any bank transaction line are correctly flagged as unmatched and not moved to the reconciled receipts folder

BE 3.3 - Description of test: Execute the reconciliation workflow and check that any receipt without a matching bank transaction line is flagged as unmatched. Make sure that it remains in the unmatched folder and is not marked as reconciled
BE 3.4 - Inputs used for test: Receipt records (receipt ID, vendor, date, amount) from matching pipeline
BE 3.5 - Expected outputs/results: Receipt flagged as unmatched in the database
BE 3.6 - Normal
BE 3.7 - White-box
BE 3.8 - Functional
BE 3.9 - Integration

BE 4.1 - Vendor Expense Sheet Export
BE 4.2 - Purpose of test: Verify that the vendor expense sheet can be exported correctly
BE 4.3 - Description of test: Request an export of the vendor expense sheet of matched data for bank statements and receipts, and check that the exported file is correct and contains the expected data
BE 4.4 - Inputs used for test: month, file format
BE 4.5 - Expected outputs/results: Exported vendor expense sheet in csv format containing information of matched expenses with correct information
BE 4.6 - Normal
BE 4.7 - Black-box
BE 4.8 - Functional
BE 4.9 - Integration

BE 5.1 - Data Export with No Available Records
BE 5.2 - Purpose of test: Verify that the backend handles export requests gracefully when no reconciliation records exist
BE 5.3 - Description of test: Submit an export request when the database has no reconciliation data, and check that the response or exported file handles this case correctly
BE 5.4 - Inputs used for test: Valid export request (month, file format) with no records in the database
BE 5.5 - Expected outputs/results: Empty export file
BE 5.6 - Boundary
BE 5.7 - Black-box
BE 5.8 - Functional
BE 5.9 - Integration

Frontend - FE (UI/UX)

UI 1.1 - Authentication Form Validation
UI 1.2 - Purpose of test: Ensure the login screen prevents submission of invalid or empty credentials.
UI 1.3 - Description of test: Attempt to click the Login button with an empty email field and a password that doesn’t match the requirements.
UI 1.4 - Inputs used for test: An empty email and Password: 123.

UI 1.5 - Expected outputs/results: A dialog box identifying a failed login attempt, no page redirection should occur.
UI 1.6 - Abnormal
UI 1.7 - Blackbox
UI 1.8 - Functional
UI 1.9 - Unit

UI 2.1 - Progress Bar Consistency
UI 2.2 - Purpose of test: Verify that the progress bar for documents in processing correctly reflects the corresponding statuses.
UI 2.3 - Description of test: Check that the colors and labels of the progress bar match the states (Green for “Complete”, red for “error”,...)
UI 2.4 - Inputs used for test: Some documents scanning of different statuses and checking the state of the progress list.
UI 2.5 - Expected outputs/results: A complete scan should display a complete, filled out green bar, while failed scans should display a red exclamation mark, and a yellow bar indicates documents still in progress.
UI 2.6 - Normal
UI 2.7 - Blackbox
UI 2.8 - Functional
UI 2.9 - Integration

UI 3.1 - Editable Export Sheet Preview
UI 3.2 - Purpose of test: Confirm that the Export Sheet allows for manual entry/edit.
UI 3.3 - Description of test: Click on a cell in any column and attempt to change the value.
UI 3.4 - Inputs used for test: Click on an empty cell with keyboard input.
UI 3.5 - Expected outputs/results: The cell should enter “edit mode” and the new value should be displayed as the user types it in.
UI 3.6 - Normal
UI 3.7 - Blackbox
UI 3.8 - Functional
UI 3.9 - Unit

UI 4.1 - Navigation Tab Active State
UI 4.2 - Purpose of test: Ensure that the app displays visually which active page they are currently viewing.
UI 4.3 - Description of test: This test is to make sure the buttons on the nav-bar will actually take the user to the intended page when they click on it.
UI 4.4 - Inputs used for test: Click on any of the items on the nav bar, including the buttons for signing out, uploading file and export sheet..
UI 4.5 - Expected outputs/results: The item on the nav bar will be highlighted and the app will display the corresponding page or dialogue option.
UI 4.6 - Normal
UI 4.7 - Blackbox

UI 4.8 - Functional
UI 4.9 - Integration

UI 5.1 - Responsive and Interactive Dashboard
UI 5.2 - Purpose of test: Make sure that when the user hovers on graphs and bars, it will display a dialogue box with more detailed explanations.
UI 5.3 - Description of test: This test is to make sure that the UI is responsive and also providing the user with more clarity on any particular details they want to know more about.
UI 5.4 - Inputs used for test: Hover mouse over any part of the dashboard.
UI 5.5 - Expected outputs/results: A dialog box will reveal itself and show a more detailed look at the section the user is hovering over.
UI 5.6 - Normal
UI 5.7 - Blackbox
UI 5.8 - Functional
UI 5.9 - Integration

Part 3. Test Case Matrix

Test Case ID	Normal/ Abnormal	Blackbox/ Whitebox	Functional/ Performance	Unit/ Integration
AIP 1	Normal	Whitebox	Functional	Unit
AIP 2	Normal	Whitebox	Performance	Unit
AIP 3	Normal	Whitebox	Functional	Unit
AIP 4	Abnormal	Whitebox	Functional	Unit
AIP 5	Abnormal	Whitebox	Functional	Unit
BE 1	Normal	Black-box	Functional	Integration
BE 2	Normal	Black-box	Functional	Integration
BE 3	Normal	White-box	Functional	Integration
BE 4	Normal	Black-box	Functional	Integration
BE 5	Boundary	Black-box	Functional	Integration
UI 1	Abnormal	Black-box	Functional	Unit

UI 2	Normal	Black-box	Functional	Integration
UI 3	Normal	Black-box	Functional	Unit
UI 4	Normal	Black-box	Functional	Integration
UI 5	Normal	Black-box	Functional	Integration